

Full Stack Development with MERN Project

Documentation format

1. Introduction

- **Project Title:** ShopEZ: E-Commerce Application
- **Team Members:** Individual

2. Project Overview

- **Purpose:** ShopEZ is a full-stack e-commerce web application designed to allow users to browse products, manage a shopping cart, and place orders online. The goal of the project is to provide a simple, user-friendly, and scalable online shopping platform for customers and administrators.
- **Features:**
 - Product listing and product details
 - Store management
 - Cart management
 - Order placement
 - Admin/seller product management
 - MongoDB database integration
 - REST API communication between frontend and backend

3. Architecture

- **Frontend:** The frontend is developed using **React.js**. It provides the user interface for customers and admins. React components are used to build pages such as home, products, cart, login, registration, and admin/store management pages. The frontend communicates with the backend using REST API calls.
- **Backend:** The backend is developed using **Node.js and Express.js**. It handles business logic, API routing, authentication, product operations, store operations, cart operations, and order management. Express routes are used to organize different modules of the application.
- **Database:** The application uses **MongoDB** as the database. MongoDB stores user details, product information, store details, cart items, and order records. The backend connects to MongoDB using a database configuration file such as server/config/db.js.

4. Setup Instructions

- **Prerequisites:** Node.js, npm, MongoDB or MongoDB Atlas, Git and Code editor such as VS Code
- **Installation:**
 - Clone the project repository:
 - Bash: `git clone <repository-url>`
 - Open the project folder:
 - Bash: `cd e-commers`
 - Install frontend dependencies:
 - Bash: `cd client`
 - `npm install`
 - Install backend dependencies:
 - Bash: `cd ../server`
 - `npm install`
 - Create a `.env` file inside the server folder:
 - Bash: `PORT=5000`
 - `MONGO_URI=<your-mongodb-connection-string>`
 - `WT_SECRET=<your-secret-key>`
 - Start MongoDB locally or make sure MongoDB Atlas is connected

5. Folder Structure

e-commers/

```
|
|
|— client/
|   |— public/
|   |— src/
|   |— index.html
|   |— package.json
|   |— eslint.config.js
|
|
|— server/
|   |— config/
|   |   |— db.js
|   |— routes/
|   |   |— storeRoutes.js
|   |— models/
|   |— controllers/
|   |— middleware/
|   |— package.json
|   |— server.js
|
|
|— README.md
```

Client:

The client folder contains the React frontend application. It includes UI components, pages, styles, and frontend configuration files.

Server:

The server folder contains the Node.js and Express backend. It includes database configuration, API routes, controllers, models, and middleware.

6. Running the Application

- Provide commands to start the frontend and backend servers locally.

Frontend: `npm run dev` (in the client directory)

Backend: `npm run dev` (in the server directory)

7. API Documentation

Method	Endpoint	Description
POST	/api/users/register	Register a new user
POST	/api/users/login	Login user
GET	/api/products	Get all products
GET	/api/products/:id	Get product details
POST	/api/products	Add new product
PUT	/api/products/:id	Update product
DELETE	/api/products/:id	Delete product
GET	/api/stores	Get all stores
POST	/api/stores	Add new store

Method	Endpoint	Description
PUT	/api/stores/:id	Update store
DELETE	/api/stores/:id	Delete store
POST	/api/cart	Add item to cart
GET	/api/cart/:userId	Get user cart
POST	/api/orders	Place order
GET	/api/orders/:userId	Get user orders

Example Response:

```
{
  "success": true,
  "message": "Product fetched successfully",
  "data": {
    "name": "Smart Watch",
    "price": 1999,
    "category": "Electronics"
  }
}
```

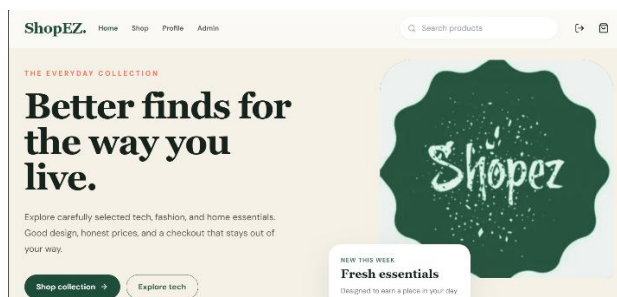
8. Authentication

Authentication is handled using email and password login. After successful login, the backend can generate a token such as a JWT token. This token is stored on the client side and sent with protected API requests.

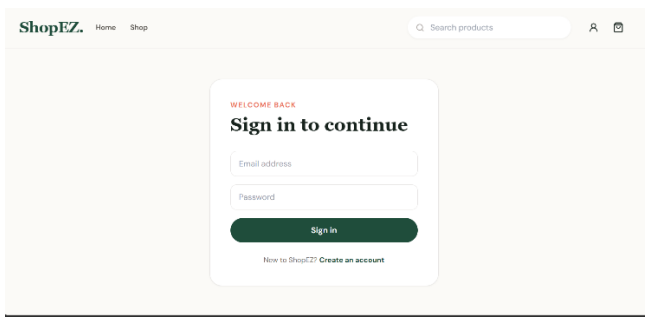
Protected routes are used for admin/seller operations such as adding, updating, or deleting products and stores. Normal users can browse products, manage cart, and place orders.

9. User Interface :-The user interface includes:

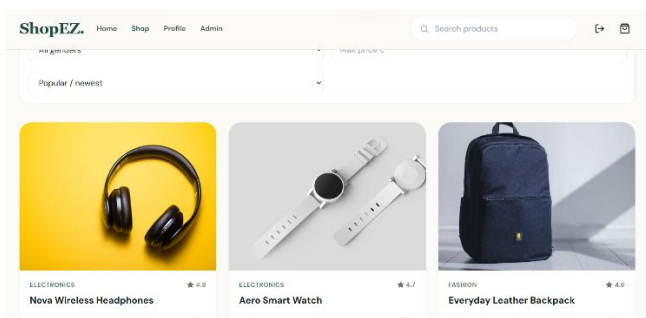
- Home page



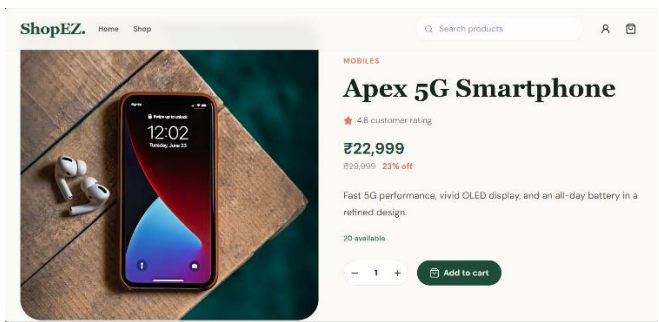
- Login page



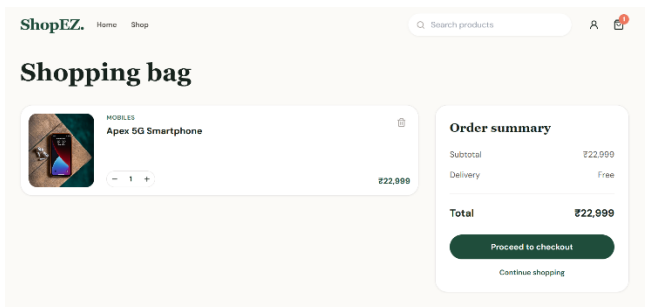
- Product listing page



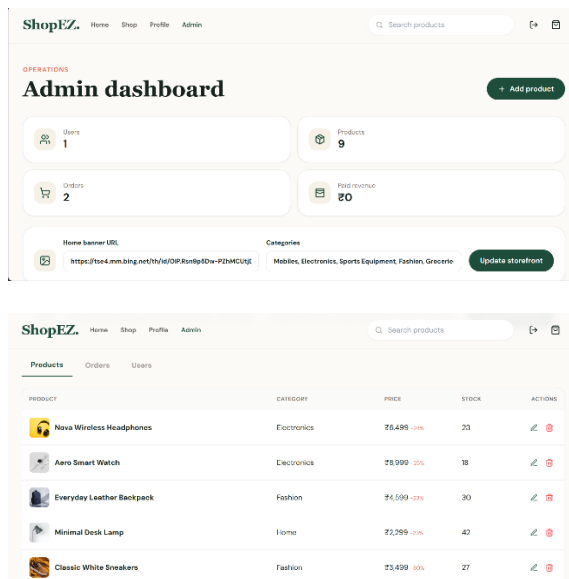
- Product details page



- Cart page



- Admin/seller management pages



10. Testing

Testing is done manually by checking the main workflows of the application.

Tested modules include:

- User registration
- User login
- Product display
- Store display
- Add to cart
- Remove from cart
- Place order
- Admin product management
- Database connection
- API responses

Tools used:

- Browser testing

- Postman for API testing
- Console/debug logs
- MongoDB database checking

11. Screenshots or Demo

Add screenshots of the following pages:

- Home page
- Login page
- Product listing page
- Cart page
- Order confirmation page
- Admin/store management page

Demo link:

<add deployed application link here>

12. Known Issues

Payment gateway is not implemented in the current version.

Email confirmation is not implemented.

Product image upload may depend on local or external image paths.

Advanced search and filtering may need improvement.

Order tracking is not available in the current version.

13. Future Enhancements

- Online payment gateway integration
- Email confirmation and order notifications

- Product reviews and ratings
- Wishlist functionality
- Advanced product search and filters
- Admin analytics dashboard
- Delivery tracking system
- Cloud image storage
- Mobile responsive improvements
- Deployment on cloud platforms like Vercel, Render, or Railway